

Phase 2: Cloud Dashboard Development

Nutritional Insights Project — CPSY 300

1. Overview

Phase 2 extends the Phase 1 diet-analysis project from a local development setup (Azure Functions Core Tools + Azurite emulator) into a fully cloud-hosted application. The dataset now lives in real Azure Blob Storage, the analysis logic runs as a live Azure Function, and the results are visualized in a browser dashboard hosted as an Azure Static Web App. The goal was to take the working Phase 1 analysis and make it publicly accessible end to end, with no local server required.

2. Architecture

The system is composed of four Azure-hosted components plus the source repository. Data flows one direction, from storage through the API to the browser:

Blob Storage → Azure Function (API) → Static Web App (dashboard) → browser charts

Component	Resource	Purpose
Storage	dietanalysisstgdhom26 (Storage Account, eastus)	Holds the "datasets" blob container with All_Diets.csv (7,806 rows).
Compute / API	diet-analysis-func-dhom26 (Function App, Linux, Python 3.11, Consumption)	HTTP-triggered Python function. Downloads the CSV from blob storage on request.
Frontend	diet-analysis-dashboard-dhom26 (Static Web App, eastus2, Free tier)	Serves a single-file HTML/JS dashboard (Chart.js) that calls the Function App API.
Source control	github.com/Dhom-10/nutritional-insights	Repository containing both the Phase 1 analysis and the Phase 2 cloud code.

API endpoints

The Function App (Python v2 programming model, `func.FunctionApp`) exposes two anonymous HTTP GET routes:

- `/api/insights` — average macronutrients per diet, top-5 protein recipes per diet, the diet with the highest average protein, most common cuisine per diet, and function execution time in ms.
- `/api/insights?diet_type=keto` — same response, filtered to a single diet type.
- `/api/diet_types` — distinct diet types, used to populate the dashboard's filter dropdown.

Every response includes `Access-Control-Allow-Origin: *` so the statically-hosted dashboard (a different origin) can call the API directly from the browser.

3. Development & Deployment Workflow

1. Provision Azure resources

Created the resource group (diet-analysis-rg, eastus) and storage account (dietanalysisstgdhom26) under the Azure for Students subscription.

2. Migrate the dataset to the cloud

Replaced Phase 1's Azurite-emulator upload script with one targeting real Blob Storage. Created the "datasets" container and uploaded All_Diets.csv (7,806 rows).

3. Build the Azure Function

Rewrote the Phase 1 analysis logic (lambda_function.py) as an HTTP-triggered Python Function using the v2 programming model. The function reads the connection string and blob names from environment variables, downloads the CSV from blob storage per request, cleans missing numeric values with column means, and computes the same insights as Phase 1.

4. Configure and deploy the Function App

Set AZURE_STORAGE_CONNECTION_STRING, BLOB_CONTAINER_NAME, and BLOB_NAME as app settings, then deployed with the Azure Functions Core Tools (func azure functionapp publish). Verified all three endpoint variations returned correct JSON, including cross-checking that keto is the highest-protein diet, matching the Phase 1 report.

5. Build the dashboard

Authored a single-file dashboard (dashboard/index.html) using Chart.js: a bar chart of average macros per diet, a doughnut chart of macro distribution, a scatter chart of top protein recipes, a top-5 protein table, a diet-type filter, a refresh button, and an execution-time readout.

6. Point the dashboard at the live API

Updated the API_BASE_URL constant in index.html to the deployed Function URL and confirmed all charts and the table render correctly against live data in a local browser.

7. Provision the Static Web App

Created diet-analysis-dashboard-dhom26 (eastus2, Free tier) and retrieved its deployment token from the Azure Portal for a CLI-based deploy.

8. Deploy and verify

Installed the SWA CLI and deployed the dashboard/ folder to the Static Web App, then confirmed the public URL loads all three charts and the table with live data.

9. Publish to GitHub

Pushed the Phase 2 code (azure-function/, dashboard/, docs/) to the project repository so the deployment is reproducible and reviewable.

4. Challenges & Solutions

Chart.js CDN blocked on local network

The `cdnjs.cloudflare.com` CDN used for Chart.js was blocked on the local network during development, so no charts rendered and the console showed failed script loads. Solved by switching the script tag to the jsdelivr CDN (`cdn.jsdelivr.net`), which was reachable.

Charts growing infinitely

The three chart canvases sat inside flexbox card containers with no explicit height, and Chart.js's responsive resize observer would grow the canvas, which grew the flex container, which grew the canvas again — an infinite feedback loop that made the page unusable. Fixed by giving each card a fixed height and setting `min-height: 0` on the canvas element (flex children default to a min-height based on content, which was defeating the fixed height).

Keeping the cloud output consistent with Phase 1

Since the Function now reads from Blob Storage instead of a local file, the results needed to match Phase 1's findings for the migration to be considered correct. Verified this directly: the deployed `/api/insights` endpoint reports keto as the diet with the highest average protein, the same conclusion as the Phase 1 report.

Per-request blob download

The Function downloads and re-parses the full CSV from Blob Storage on every request rather than caching it, which is simple but adds latency. This was an acceptable trade-off for a course-scale dataset (7,806 rows) and a Consumption-plan Function; the dashboard's execution-time readout makes this cost visible rather than hiding it.

5. Deliverables

Deliverable	Link / Status
Azure Function API (live)	https://diet-analysis-func-dhom26.azurewebsites.net/api/insights
Azure Static Web App (dashboard)	https://red-cliff-0a1e5190f.7.azurestaticapps.net (deploy via SWA CLI to go live)
GitHub repository	github.com/Dhom-10/nutritional-insights (push Phase 2 code to publish)
This document	Phase2_Documentation.pdf

Note: the Static Web App deploy and GitHub push were the two remaining steps as of this writing; both are included above with the URLs already provisioned so this table can be updated in place once each finishes.